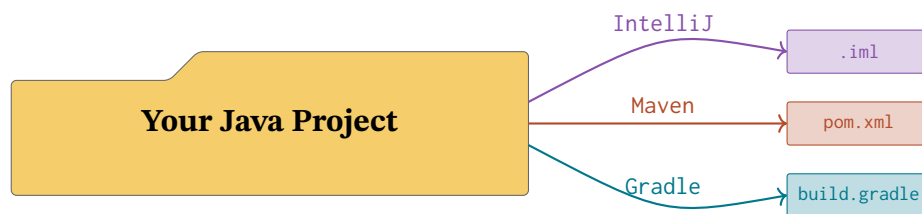


A Student's Handbook to the JDK, Java Projects, and Java Build Systems

A practical, picture-first guide for SWE-4202



Course: Object Oriented Concepts I

*Designed so that you never have to ask:
“Why won’t my JUnit tests run?” or “Where do dependencies live?” again.*

Contents

1	Why This Handbook Exists	2
2	What Is the JDK?	2
2.1	The Three Pieces You Will Use Daily	3
2.2	JDK Versions and Why They Matter	3
3	What Is a Java Project, as a Folder?	3
4	What Is a “Build System” and Why Do We Need One?	4
5	The Three Build Setups You Will See in This Course	5
5.1	Setup 1 — The IntelliJ “Build System”	5
5.1.1	When to Use IntelliJ-Only Projects	6
5.2	Setup 2 — Maven	6
5.2.1	The Maven Folder Layout	6
5.2.2	Anatomy of a pom.xml	7
5.2.3	Common Maven Commands	8
5.3	Setup 3 — Gradle	8
5.3.1	The Gradle Folder Layout	9
5.3.2	Anatomy of a build.gradle	9
5.3.3	The Gradle Wrapper (gradlew)	10
6	How to Identify a Project’s Build System at a Glance	11
6.1	Side-by-Side: One Project, Three Build Systems	11
7	Reading the Two Demo Projects	11
7.1	untitled1/ — the Maven version	11
7.2	untitled/ — the Gradle version	11
7.3	Adding a Real Dependency Without Touching IntelliJ	12
8	The Files Each Build System Must Have	12
9	Mastering .gitignore	13
9.1	What .gitignore Actually Is	13
9.2	Pattern Syntax in 60 Seconds	14
9.3	What to Always Ignore in Any Java Project	14
9.4	Per-Project-Type Recipes	14
9.4.1	Recipe A — IntelliJ-only Project (Lab 03 RideNow-style)	14
9.4.2	Recipe B — Maven Project (pom.xml present)	15
9.4.3	Recipe C — Gradle Project (build.gradle present)	16
9.5	Decision Cheat-Sheet	17
9.6	Generating a .gitignore Automatically	17
10	Where Dependencies Actually Live on Your Computer	18
11	Quick Reference Card	19
12	What Comes Next	19
	Appendix: Why We Voluntarily Suffer	20

1. Why This Handbook Exists

In Lab 03 of this course you wrote Java classes inside the RideNow project. That project was opened directly in IntelliJ IDEA and used what we will call the *IntelliJ build system*. Whenever you wanted to run a JUnit test, you had to manually attach the JUnit 5 library to the project's classpath. Some of you did this once and forgot how. Some of you did it three times before it worked. All of you, at some point, asked: “Where does this JUnit library come from, and why isn't it just there?”

That is the right question. From the next labs onward, we are going to switch to a **proper build system** (Maven or Gradle). Once we do, JUnit, Mockito, JaCoCo, and any other library you might need will appear with a single line of configuration — not by hunting JAR files on your computer.

But before we make that switch, you need to understand what is actually happening when you “run a Java program.” This handbook walks you through:

- What the **JDK** is and how the pieces (javac, java, JVM, JRE) fit together.
- What a **Java project** looks like as a folder on disk.
- What a **build system** does and why every serious Java project uses one.
- The three build setups you will encounter in this course — **IntelliJ**, **Maven**, and **Gradle** — with the exact files each one requires.
- How to **identify** which build system a project uses just by looking at its folder.
- How **dependencies** (e.g. JUnit 5) are declared in each system.

Note

This handbook is a companion to *A Student's Handbook to Git, GitHub, and the Course Folder Structure*. That one taught you how to navigate the course *repository*. This one teaches you how to navigate any *project* you find inside it.

2. What Is the JDK?

When people say “install Java,” they almost always mean “install the **JDK**.” The JDK is a bundle of programs and libraries that lets you both *write* Java code and *run* Java code. The bundle includes three things you will hear about repeatedly: javac, the java launcher, and the JVM.

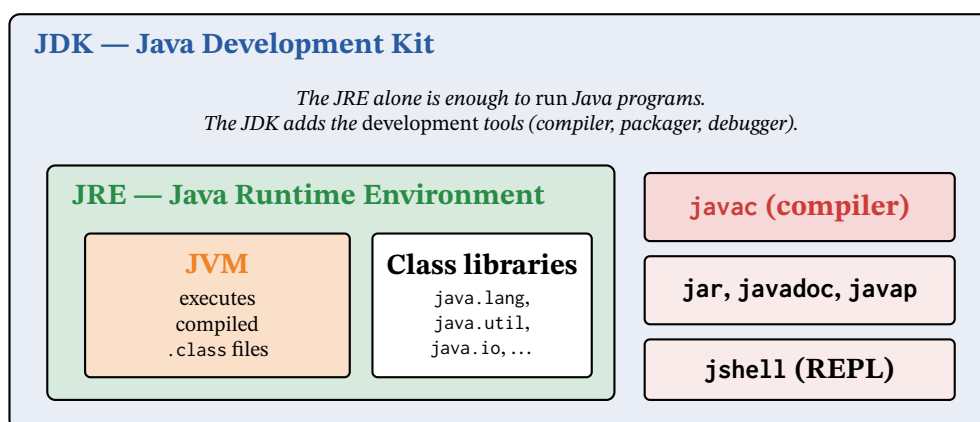


Figure 1: The JDK is a superset of the JRE, which is a superset of the JVM.

2.1. The Three Pieces You Will Use Daily

1. **javac — the compiler.** It takes .java text files and produces .class bytecode files.
2. **java — the launcher.** It starts the JVM and tells it which .class file's main method to run.
3. **The JVM (Java Virtual Machine)** — a program that reads .class files and executes them. The JVM is what makes Java “write once, run anywhere”: the bytecode is the same on Windows, macOS, and Linux; only the JVM is different.

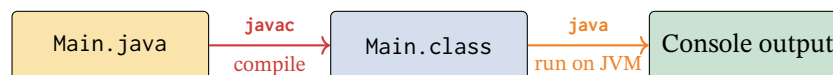


Figure 2: The two-step lifecycle of any Java program: compile, then run.

Command: javac & java

```
# 1. Compile your source file -> produces Main.class
javac Main.java

# 2. Run the bytecode on the JVM
java Main
```

2.2. JDK Versions and Why They Matter

The JDK is versioned: JDK 8, JDK 11, JDK 17, JDK 21, and so on. The version of the JDK you compiled with determines which language features you can use (e.g. var, records, switch expressions). It also determines the minimum JVM version that can run your code. In this course we target **JDK 17** or newer.

Note

You can have several JDKs installed at once. IntelliJ, Maven, and Gradle each let you point a project at a specific JDK without changing your system-wide “default” JDK. This is convenient: a Lab 02 project written for JDK 8 can coexist with a Lab 07 project written for JDK 21.

Do This

Check what you have right now by opening a terminal and running `java -version` and `javac -version`. They should print the same major version. If they do not, your shell is finding the JRE-only java from one place and the JDK javac from another — a classic source of confusion.

3. What Is a Java Project, as a Folder?

A *Java project* is just a directory on disk that follows a few conventions so that tools (your IDE, a build system) can find:

- your **source code** (.java files),
- your **tests** (also .java files, but treated separately),

- your **compiled output** (.class files; sometimes packaged into a .jar),
- your **dependencies** (the libraries you depend on, like JUnit),
- and a **configuration file** that ties everything together (.iml, pom.xml, or build.gradle).

The configuration file is what tells you which *build system* the project uses. Once you can spot the configuration file, you can identify any Java project at a glance.

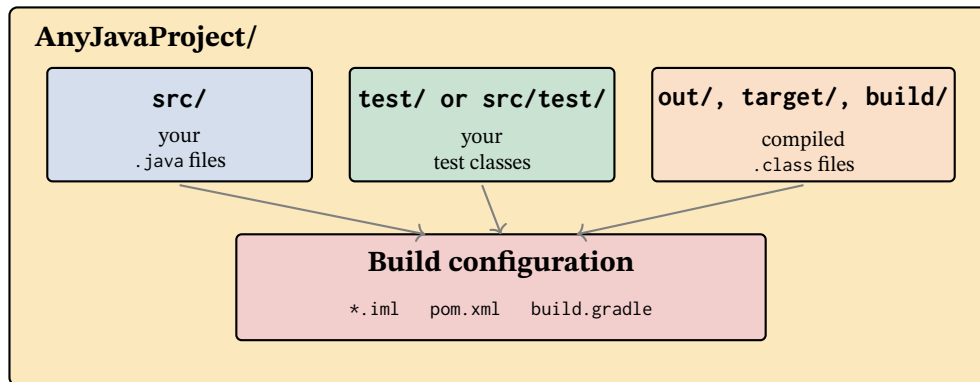


Figure 3: Every Java project has the same conceptual parts. Only the names and the configuration file change.

4. What Is a “Build System” and Why Do We Need One?

In Lab 02 and Lab 03 you compiled and ran code with a single click in IntelliJ. You may not have realized it, but a lot was happening behind the scenes. *Building* a Java project means at minimum:

1. Finding every .java source file.
2. Putting every required library on the classpath.
3. Calling javac on the sources.
4. Putting the resulting .class files in an output directory.
5. Running the test classes against the compiled output.
6. (Optionally) packaging the result into a .jar that can be shipped.

Doing this by hand is tedious. Doing it consistently across machines (yours, your teammate’s, the lab computer’s) is even harder. A **build system** is a tool that automates these steps using a single configuration file checked into Git. With it, anyone can clone your repository and run *one command* to get the same compiled, tested artifact you have.

Why It Matters

The reason JUnit was so painful in Lab 03 is that the IntelliJ-only project did not have a build system in the modern sense. The list of libraries lived inside an IDE-specific file (RideNow.iml), so anyone opening the project on a different machine had to re-attach JUnit themselves. With Maven or Gradle, the line “testImplementation 'org.junit.jupiter:junit-jupiter'” (or its XML equivalent) is enough — the build system downloads JUnit automatically the first time you build.

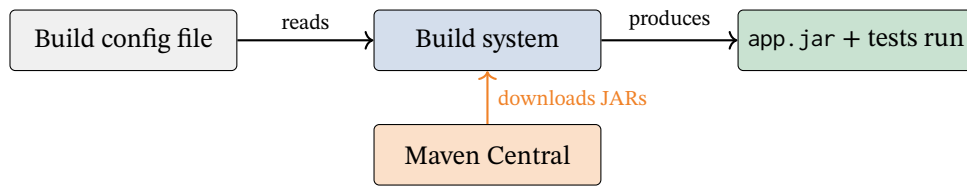


Figure 4: A build system is the bridge between a small text configuration file and a fully built, tested project.

5. The Three Build Setups You Will See in This Course

5.1. Setup 1 — The IntelliJ “Build System”

In Lab 02 and Lab 03 your project (RideNow) was managed entirely by IntelliJ. There was no `pom.xml` and no `build.gradle`; instead, IntelliJ stored the project’s settings in two places:

- A `.idea/` folder (project-wide settings: code style, modules, run configurations).
- A `*.iml` file — one per module — listing the source roots and the libraries.

For RideNow, the `.iml` looked roughly like this:

Listing 1: RideNow.iml — the IntelliJ module file. Note that the JUnit 5 library is hard-wired to a specific path inside your local Maven cache.

```

<?xml version="1.0" encoding="UTF-8"?>
<module type="JAVA_MODULE" version="4">
  <component name="NewModuleRootManager" inherit-compiler-output="true">
    <exclude-output />
    <content url="file://$MODULE_DIR$">
      <sourceFolder url="file://$MODULE_DIR$/src" isTestSource="false" />
      <sourceFolder url="file://$MODULE_DIR$/test" isTestSource="true" />
    </content>
    <orderEntry type="inheritedJdk" />
    <orderEntry type="sourceFolder" forTests="false" />
    <orderEntry type="module-library" scope="TEST">
      <library name="JUnit5">
        <CLASSES>
          <root url="jar://$MAVEN_REPOSITORY$/org/junit/jupiter/.../junit-
            jupiter-5.14.0.jar!/" />
          ...
        </CLASSES>
      </library>
    </orderEntry>
  </component>
</module>
  
```

Folder layout you used in Lab 03 (RideNow):

```

RideNow/
|-- .idea/                <- IntelliJ project settings (do NOT
    edit by hand)
|-- RideNow.iml           <- the IntelliJ module file
|-- src/                  <- production sources (flat, no
    package folders)
|   |-- Driver.java
|   |-- Location.java
|   |-- Main.java
|   |-- Passenger.java
|   `-- Trip.java
|-- test/                  <- test sources (also flat)
|   `-- RideNowTest.java
|-- out/                   <- compiled .class files (auto-
    generated)
`-- .gitignore

```

Common Mistake

The `.iml` file is IDE-specific. It works only inside IntelliJ, and it stores library paths that are valid only on *your* machine. The moment you collaborate with someone else — or move to another computer — those library paths break. That is exactly the pain you felt with JUnit in Lab 03.

5.1.1. When to Use IntelliJ-Only Projects

This setup is acceptable when:

- You are doing a tiny throw-away exercise (a few classes, no libraries).
- You are working alone on one machine.
- You have no tests beyond what JUnit ships with IntelliJ.

For everything else — including the rest of this course — you will use Maven or Gradle.

5.2. Setup 2 — Maven

Apache Maven is the older and more conservative of the two “real” build systems. It is built around a strict, opinionated convention: every Maven project follows the same folder structure, runs the same lifecycle (`validate` → `compile` → `test` → `package` → `install`), and has exactly one configuration file: `pom.xml` (*Project Object Model*).

5.2.1. The Maven Folder Layout

Maven imposes a directory layout. Every Maven project on Earth looks like this:

```

untitled1/
|-- pom.xml          <- the Maven project folder
|-- .mvn/            <- THE configuration file
    config          <- (optional) Maven Wrapper
|-- src/
    |-- main/
        |-- java/    <- production source code
            |-- org/example/
            |-- Main.java
        |-- resources/ <- non-code assets (config
            files, etc.)
        |-- test/
            |-- java/ <- test source code
                |-- org/example/
                |-- CalculatorTest.java
    |-- target/      <- compiled output (auto-
        generated)

```

Note

The deeply nested `src/main/java/org/example/` path is not arbitrary. The `src/main/java` prefix is a Maven convention; the `org/example` part *must* match the package declared at the top of your `Main.java` (`package org.example;`). One folder per package segment — Java's package rules force this.

5.2.2. Anatomy of a pom.xml

Below is the actual `pom.xml` from the demo project `untitled1/` that ships with this handbook. It is intentionally minimal but already does everything an IntelliJ-only project could not:

Listing 2: `untitled1/pom.xml` — a complete, working Maven configuration with JUnit 5.

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>untitled1</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <junit.version>5.10.2</junit.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>

```

```

        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.2.5</version>
            </plugin>
        </plugins>
    </build>
</project>

```

The four blocks you must recognize:

1. **Project coordinates** (<groupId>, <artifactId>, <version>). These three together uniquely identify your project. groupId is your organization (often a reverse domain like org.example), artifactId is the project name, and version is, well, the version.
2. **<properties>**. Key-value pairs reused below. Here we lock the Java compiler to JDK 17 and store the JUnit version once.
3. **<dependencies>**. The libraries you need. Each dependency is identified by its own groupId / artifactId / version. Maven will fetch the JAR (and any libraries *it* depends on) from *Maven Central* the first time you build.
4. **<build><plugins>**. Plugins extend Maven. maven-surefire-plugin is what actually runs your tests during the test phase.

5.2.3. Common Maven Commands

Command: mvn

```

mvn compile           # compile src/main/java
mvn test              # compile + run tests in src/test/java
mvn package           # compile + test + produce target/untitled1
-1.0-SNAPSHOT.jar
mvn clean             # delete the target/ folder
mvn clean install     # rebuild everything from scratch and install
to local repo

```

Do This

The first time you run `mvn test`, Maven will download JUnit (and its dependencies) from Maven Central into a hidden cache at `~/.m2/repository`. After that, builds are fast because the JARs are already on disk.

5.3. Setup 3 — Gradle

Gradle is the newer of the two. Where Maven uses XML and a fixed lifecycle, Gradle uses a small *Groovy* (or *Kotlin*) script and a flexible task graph. The result is that Gradle build files are usually much shorter than the Maven equivalent — but they have a learning curve when you start customizing them.

5.3.1. The Gradle Folder Layout

Gradle adopted Maven's directory convention, so the source tree looks identical. The configuration files are different, though:

```

untitled/                                <- the Gradle project
  folder
|-- build.gradle                          <- THE configuration file
|-- settings.gradle                      <- root project name and
   included modules
|-- gradlew                              <- Gradle Wrapper (Linux/
   macOS)
|-- gradlew.bat                          <- Gradle Wrapper (Windows)
|-- gradle/
|   |-- wrapper/
|   |   |-- gradle-wrapper.jar
|   |   |-- gradle-wrapper.properties <- which Gradle version
|   |   to use
|-- src/
|   |-- main/
|   |   |-- java/
|   |   |   |-- org/example/
|   |   |   |-- Main.java
|   |   |-- resources/
|   |-- test/
|   |   |-- java/
|   |   |   |-- org/example/
|   |   |   |-- ExampleTest.java
|   |   |-- resources/
|-- build/                               <- compiled output (auto-
   generated)

```

5.3.2. Anatomy of a build.gradle

Compare this to the Maven pom.xml above — same project, same dependency, less ceremony:

Listing 3: untitled/build.gradle (text version of Figure 5).

```

plugins {
    id 'java'
}

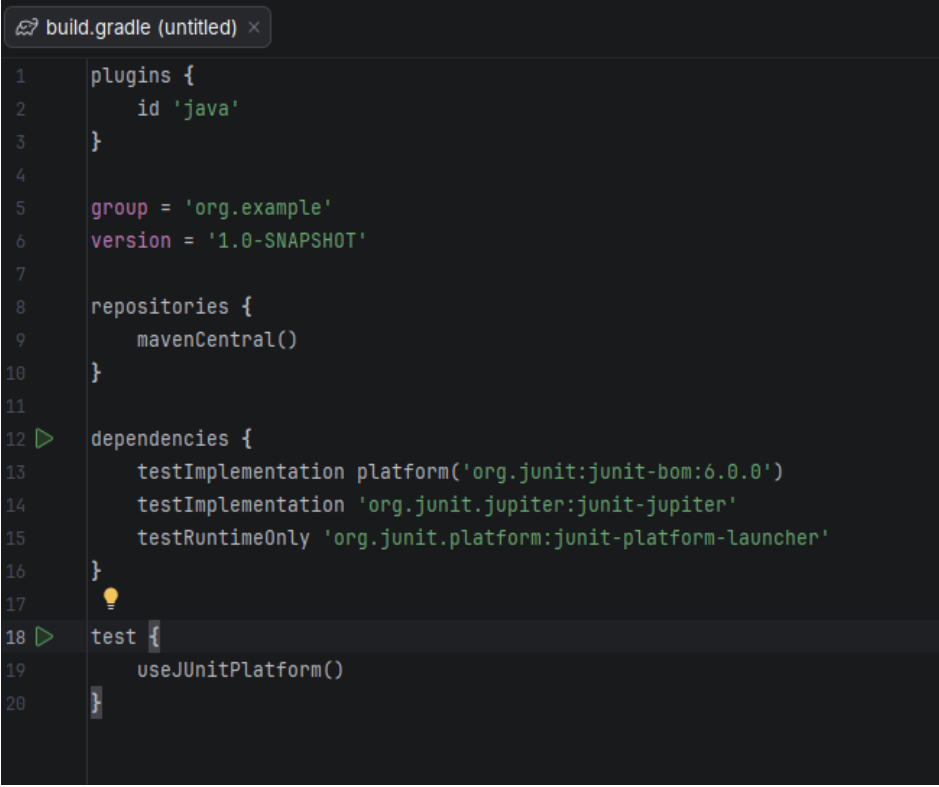
group = 'org.example'
version = '1.0-SNAPSHOT'

repositories {
    mavenCentral()
}

dependencies {
    testImplementation platform('org.junit:junit-bom:6.0.0')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

test {
    useJUnitPlatform()
}

```

A screenshot of the IntelliJ IDEA IDE showing a file named 'build.gradle (untitled)'. The code is as follows:

```
1  plugins {  
2      id 'java'  
3  }  
4  
5  group = 'org.example'  
6  version = '1.0-SNAPSHOT'  
7  
8  repositories {  
9      mavenCentral()  
10 }  
11  
12 dependencies {  
13     testImplementation platform('org.junit:junit-bom:6.0.0')  
14     testImplementation 'org.junit.jupiter:junit-jupiter'  
15     testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
16 }  
17  
18 test {  
19     useJUnitPlatform()  
20 }
```

Figure 5: The full build.gradle from the demo project untitled/, opened in IntelliJ IDEA. Twenty lines is enough to declare the language, the dependencies, and the test runner.

```
}
```

The five blocks you must recognize:

1. **plugins { id 'java' }**. Tells Gradle this is a Java project (and gives it tasks like compileJava, test, jar).
2. **group and version**. The Gradle equivalent of Maven's groupId / version. The folder name is the implicit artifactId (overridable in settings.gradle).
3. **repositories**. Where to look for dependencies. mavenCentral() is the same Maven Central repository Maven uses.
4. **dependencies**. The library list. testImplementation is Gradle-speak for “only on the test class-path”; it's the same idea as Maven's <scope>test</scope>.
5. **test { useJUnitPlatform() }**. Tells Gradle's built-in test task to drive JUnit 5 (the “Jupiter” platform).

5.3.3. The Gradle Wrapper (gradlew)

You may have noticed two extra files at the project root: gradlew and gradlew.bat. These are the *Gradle Wrapper*. They let anyone build the project with the *exact* Gradle version it was written for, even if they have no Gradle installed system-wide.

Command: ./gradlew

```

./gradlew build      # compile + test + package
./gradlew test       # only run tests
./gradlew clean      # delete the build/ folder
./gradlew tasks      # list every task this project knows about

```

Do This

Always prefer `./gradlew` (Linux/macOS) or `gradlew.bat` (Windows) over a globally installed `gradle`. The wrapper version is pinned in `gradle/wrapper/gradle-wrapper.properties` and committed to Git, so every collaborator builds with the same Gradle.

6. How to Identify a Project's Build System at a Glance

The single quickest skill you can build is recognizing a project's build system from its folder. Use this decision tree:

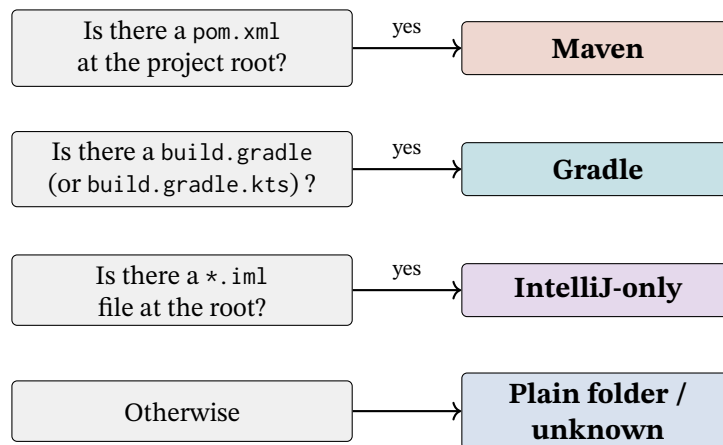


Figure 6: A 5-second decision tree for identifying any Java project.

6.1. Side-by-Side: One Project, Three Build Systems

7. Reading the Two Demo Projects

Inside `Lab-03/` you will find two folders: `untitled/` (Gradle) and `untitled1/` (Maven). They are tiny and exist only so you can compare them directly.

7.1. `untitled1/` — the Maven version

- Has a `pom.xml` at the root — so this is Maven.
- Source code lives at `src/main/java/org/example/Main.java`.
- The `target/` folder is created when you run `mvn compile`; do not commit it to Git.

7.2. `untitled/` — the Gradle version

- Has `build.gradle` and `settings.gradle` at the root — so this is Gradle.

	IntelliJ-only	Maven	Gradle
Config file	*.iml, .idea/	pom.xml	build.gradle, settings.gradle
Config syntax	XML (IDE-managed)	XML	Groovy (or Kotlin) DSL
Source folder	src/ (flat)	src/main/java/	src/main/java/
Test folder	test/	src/test/java/	src/test/java/
Output folder	out/	target/	build/
Add a library	Manually attach JAR in IDE	Add <dependency> in pom.xml	Add line in dependencies { }
Build command	(run via IntelliJ)	mvn package	./gradlew build
Portable across machines?	No	Yes	Yes

Table 1: Every Java project has the same parts. The build system decides what they are called and how they are configured.

- Same source layout as Maven (src/main/java/org/example/Main.java).
- Ships with the wrapper (gradlew, gradlew.bat, gradle/wrapper/). Use those.
- The build/ folder is created when you run ./gradlew build; do not commit it.

Note

Open both Main.java files and you will see they are byte-for-byte identical. The Java source code does not care which build system surrounds it. That is the whole point of the build system: it gives you tools and dependency management without changing how you write Java.

7.3. Adding a Real Dependency Without Touching IntelliJ

To prove the difference from Lab 03, try adding a tiny library — say, Apache Commons Lang — to untitled1/ (the Maven project). All you need is to add this block inside <dependencies>:

```
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-lang3</artifactId>
  <version>3.14.0</version>
</dependency>
```

Save the file, run `mvn compile`, and the JAR is fetched and put on the classpath. You did not open IntelliJ. You did not download anything by hand. That is the whole game.

8. The Files Each Build System Must Have

Use this checklist as a reference. If a file is missing, the corresponding build will fail.

File / folder	Purpose	System
*.iml	Module definition (sources, libraries)	I
.idea/	IDE-wide project settings	I
pom.xml	Project Object Model: coordinates, dependencies, plugins	M
src/main/java/	Production source code	M, G
src/main/resources/	Non-code resources (.properties, etc.)	M, G
src/test/java/	Test source code	M, G
target/	Maven build output (<i>git-ignored</i>)	M
build.gradle	Gradle build script (Groovy DSL)	G
settings.gradle	Defines the root project name and any subprojects	G
gradlew, gradlew.bat	Gradle Wrapper launchers	G
gradle/wrapper/gradle-wrapper.properties	Pins the Gradle version to use	G
build/	Gradle build output (<i>git-ignored</i>)	G

Table 2: What each build system needs. **I** = IntelliJ-only, **M** = Maven, **G** = Gradle.

9. Mastering .gitignore

A clean repository is a repository that contains *only the things you intend to commit*. No compiled .class files, no build outputs that will be different on every machine, no IDE caches, no OS clutter like .DS_Store. The tool that gets you there is a tiny text file called .gitignore.

9.1. What .gitignore Actually Is

.gitignore is a plain-text file — usually placed at the root of a project — that lists **patterns**. Any file whose path matches a pattern is invisible to Git: it will not show up in `git status`, it will not be staged by `git add .`, and it will never end up in a commit. It is your way of saying “this file is allowed to exist on disk, but Git should pretend it doesn’t.”

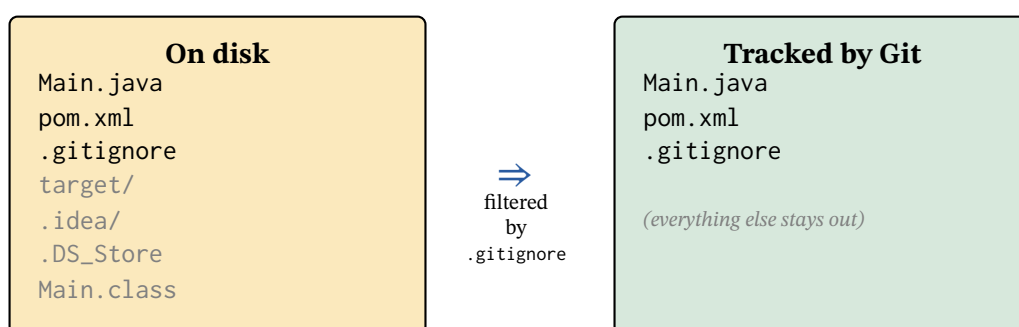


Figure 7: .gitignore sits between your working folder and what actually ends up in commits.

Pattern	Meaning
target/	Ignore any folder named target anywhere in the project.
/target	Ignore target only at the project root.
.class	Ignore every file ending in .class.
*/build/	Ignore every build/ folder, at any depth.
!gradle-wrapper.jar	<i>Un-ignore</i> this file even if a previous rule matched it.
# this is a comment	Lines starting with # are comments.

Table 3: The five patterns that cover 95% of real-world .gitignore files.

9.2. Pattern Syntax in 60 Seconds

Note

.gitignore only affects files that are *not yet tracked*. If you accidentally committed target/ once and then added it to .gitignore, Git will keep tracking it. Run `git rm -r --cached target/` to stop tracking it; the files stay on disk, but Git lets go of them.

9.3. What to Always Ignore in Any Java Project

Regardless of build system, these belong in every Java project's .gitignore:

Listing 4: Universal Java ignore rules — safe to copy into any project.

```
# Compiled bytecode
*.class

# Logs
*.log

# OS noise
.DS_Store          # macOS
Thumbs.db          # Windows
desktop.ini         # Windows

# Editor backups
*~
*.swp
```

9.4. Per-Project-Type Recipes

The interesting question is: *what should I add for my build system?* Use the recipe that matches the configuration file you found in Section 6.

9.4.1. Recipe A — IntelliJ-only Project (Lab 03 RideNow-style)

This is the trickiest case, and it is also the one that caused you the most pain in Lab 03. In an IntelliJ-only project, the .iml file and the .idea/ folder *are* the project definition — delete them and there is nothing left for IntelliJ to open. So you need to be careful what you ignore.

IntelliJ-only .gitignore

```
# Compiled output (regenerated on every build)
out/
*.class

# IntelliJ user-specific state (do NOT commit)
.idea/workspace.xml
.idea/tasks.xml
.idea/usage.statistics.xml
.idea/shelf/
.idea/dictionaries/

# OS noise
.DS_Store
Thumbs.db
```

Notice what is *not* ignored: *.iml, .idea/modules.xml, .idea/misc.xml, and .idea/compiler.xml. These describe the project's source roots and module structure — without them, your collaborators see an empty IntelliJ window when they clone.

Common Mistake

The .gitignore that shipped with Lab-03/RideNow ignored *all* of .idea/ and *all* *.iml files. That is the structural reason JUnit 5 had to be re-attached on every machine: the library reference lived in RideNow.iml, which Git was throwing away. The fix in Lab 03 was to attach JUnit by hand each time. The fix from the next lab onward is simpler — use Maven or Gradle, where the dependency lives in a build file that *is* committed.

9.4.2. Recipe B — Maven Project (pom.xml present)

Now the rules become clean and bulletproof: anything IntelliJ generates is regenerated from pom.xml, so you can ignore it freely.

Maven .gitignore

```
# Maven build output
target/

# Maven release/backup files
pom.xml.tag
pom.xml.releaseBackup
pom.xml.versionsBackup
pom.xml.next
release.properties

# IntelliJ (regenerated from pom.xml on import)
.idea/
*.iml
*.iws
*.ipr
out/

# Eclipse
.classpath
.project
.settings/

# VS Code
.vscode/

# Compiled bytecode
*.class

# OS noise
.DS_Store
Thumbs.db
```

What you commit: `pom.xml`, everything under `src/`, the `.mvn/` folder if present (it pins the Maven Wrapper version), and your own `.gitignore`.

9.4.3. Recipe C — Gradle Project (`build.gradle` present)

Gradle's ignore list is similar but adds two Gradle-specific folders.

Gradle .gitignore

```
# Gradle build output and cache
build/
.gradle/

# Keep the Gradle Wrapper JAR even though *.jar is often ignored
# elsewhere
!gradle/wrapper/gradle-wrapper.jar

# IntelliJ (regenerated from build.gradle on import)
.idea/
*.iml
*.iws
*.ipr
out/

# Eclipse / VS Code
.classpath
.project
.settings/
.vscode/

# Compiled bytecode
*.class

# OS noise
.DS_Store
Thumbs.db
```

What you commit: build.gradle, settings.gradle, everything under src/, the entire gradle/wrapper/ folder, both gradlew and gradlew.bat, and your .gitignore.

Do This

The !gradle/wrapper/gradle-wrapper.jar line uses a negation pattern. It says: “even if some other rule (or a global ignore) would skip this JAR, keep it.” Without that line, anyone who clones the repository would not be able to run ./gradlew on their machine.

9.5. Decision Cheat-Sheet

9.6. Generating a .gitignore Automatically

You don’t have to write these from memory. Two reliable sources:

- **gitignore.io** — type “Java, Maven, IntelliJ” (or whichever combination matches your project) and it generates a complete file you can paste in.
- **github/gitignore** on GitHub — the canonical templates IntelliJ, Maven, and Gradle ship with. The two demo projects in this folder (untitled/ and untitled1/) use these templates as their starting point.

File / folder	IntelliJ-only	Maven	Gradle
*.class	ignore	ignore	ignore
out/	ignore	ignore	ignore
target/	—	ignore	—
build/	—	—	ignore
.gradle/	—	—	ignore
*.iml	commit	ignore	ignore
.idea/ (whole folder)	partial [†]	ignore	ignore
pom.xml	—	commit	—
build.gradle, settings.gradle	—	—	commit
gradle/wrapper/gradle-wrapper.jar	—	—	commit
src/	commit	commit	commit
.DS_Store, Thumbs.db	ignore	ignore	ignore

Table 4: [†]For an IntelliJ-only project: ignore .idea/workspace.xml, .idea/tasks.xml, .idea/shelf/, .idea/usage.statistics.xml, .idea/dictionaries/; commit .idea/modules.xml, .idea/misc.xml, .idea/compiler.xml.

Do This

When you create a new repository on GitHub, the “Add .gitignore” dropdown lets you pick a template (Java, Maven, Gradle, etc.) at creation time. Use it. It is far easier to start with a good .gitignore than to clean up an accidentally-committed target/ folder later.

10. Where Dependencies Actually Live on Your Computer

When Maven or Gradle “downloads JUnit,” the JAR file ends up in a real folder on your hard drive — you can open it and see it. Knowing where they live makes things less mysterious.

- **Maven local cache:** ~/.m2/repository/
Inside, the JUnit JAR is at
~/.m2/repository/org/junit/jupiter/junit-jupiter/5.10.2/junit-jupiter-5.10.2.jar.
- **Gradle local cache:** ~/.gradle/caches/modules-2/files-2.1/
Same JAR, different folder layout.

Note

The IntelliJ-only RideNow.iml file you saw in Lab 03 actually *pointed at* files in ~/.m2/repository/. That is why JUnit “worked” on your machine: someone, somewhere along the way, had run a Maven command that populated that cache. On a freshly installed machine, the same .iml would have failed.

11. Quick Reference Card

Cheat sheet — pin this to your monitor

Identify the project

- `pom.xml` present ⇒ Maven.
- `build.gradle` present ⇒ Gradle.
- Only `*.iml` + `.idea/` ⇒ IntelliJ-only.

Open in IntelliJ IDEA

- Always open the folder that *contains* the configuration file. Not the parent, not the lab folder.

Build from the terminal

- Maven: `mvn clean test`
- Gradle: `./gradlew clean test`

Add a dependency

- Maven: add a `<dependency>` block to `pom.xml`.
- Gradle: add a line under dependencies `{ . . . }` in `build.gradle`.
- IntelliJ-only: *don't*. Move the project to Maven or Gradle first.

12. What Comes Next

From the next lab onward, every project handed out in this course will ship as a Maven or Gradle project. The first thing you should do when you open one is:

1. Look at the root: spot `pom.xml` or `build.gradle`.
2. Open *that* folder in IntelliJ IDEA. The IDE will detect the build system, import the project, and download dependencies on its own. You should see a progress bar at the bottom of the window the first time.
3. In the terminal, run `mvn test` or `./gradlew test`. If the tests run and pass, your environment is correctly set up.

Do This

The day you write your first JUnit test in a Maven or Gradle project and it just runs — with no library hunt, no missing classpath errors, no IntelliJ pop-ups — you will understand why we are making this move. Welcome to professional Java.

Appendix: Why We Voluntarily Suffer

A Frank Word on Why Java Projects Are Harder to Keep Tidy Than C# Projects

Somewhere in another classroom, on another floor, the C# students are having a much calmer afternoon. They opened Visual Studio. It asked them, politely, if they would like a project. They said yes. A single `.csproj` file appeared — twelve lines of XML, all of it human-readable. They typed `dotnet add package xUnit`. NuGet did exactly what its name promises. Their tests ran. They went for tea.

We, meanwhile, are on our third Stack Overflow tab trying to figure out why `junit-jupiter-engine` is on the compile classpath but somehow not on the test runtime classpath, and whether the answer is a Maven *scope*, a Gradle *configuration*, an IntelliJ *module library*, or simply punishment for our sins.

A short tour of the things C# does not make you think about

- **One project file, not three.** A C# project is a `.csproj`. That's it. No `pom.xml` and `settings.xml` and `.mvn/`, no `build.gradle` and `settings.gradle` and `gradle-wrapper.properties`. Just one file. A child could read it. Many children have.
- **No wrapper scripts.** The `dotnet` CLI is installed once and works for every project. We, on the other hand, ship a 60 KB JAR called `gradle-wrapper.jar` *inside the repository* so that the build tool can bootstrap the build tool. This is not a joke. It is in your `<ProjectName>/gradle/wrapper/` folder. Go look.
- **Properties.** In C#, `public string Name { get; set; }` is one line. In Java, the same thing requires a private field, a getter, a setter, your IDE's "Generate" menu, a small prayer, and a bigger equals/hashCode dialog two minutes later. Lombok exists, but using Lombok requires installing Lombok, which requires explaining Lombok to the next person, which is its own form of suffering.
- **NuGet is one place. "Java dependencies" is six.** C# people fetch packages from NuGet. Java packages can come from Maven Central, JCenter (rest in peace), JitPack, your company's Nexus, your company's Artifactory, an unsigned ZIP your senior dropped in Slack, or a `lib/` folder that someone committed in 2014. Every one of those is a different way for the build to break.
- **Async.** C# has `async/await`. Java has `CompletableFuture.thenComposeAsync(...).exceptionallyCompose(...)` and a stack trace that starts at line 1 of a lambda you don't remember writing.
- **Null.** C# has nullable reference types built into the compiler. Java has `Optional`, which everyone agrees is great, except that half the standard library still returns `null`, so you wrap it, and then someone unwraps it with `.get()`, and then the universe restores balance with a `NullPointerException`.
- **LINQ.** They get `users.Where(u => u.IsActive).OrderBy(u => u.Name)`. We get `users.stream().filter(u -> u.isActive()).sorted(Comparator.comparing(User::getName)).collect(Collectors.toList())`. Both compile. Only one of them will fit on a slide.

— The course staff, who have made peace with their classpath